

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Dot .Net Core and Progressive Web Application Practical

Practical – II

Roll No.: T005	Name: Ritesh Chaubey
Class: TYIT	Batch: 1
Date of Assignment: 26-06-2025	Date/Time of Submission: 26-06-2025

PART – A

Write the program with different features of C#

- a) Function Overloading
- b) Constructor overloading
- c) Inheritance (all types)
- d) Interfaces
- e) Using Delegates and events
- f) Exception handling

PART B :

Solution:

A) CODE

```
using System;

public class ABC{

    void Sun1(int num1, int num2){

        Console.WriteLine("Subtraction of Two Numbers : " + (num1 - num2));

    }

    void Sun1(int num, int num1, int num2){

        Console.WriteLine("Addition of 3 Numbers : " + (num + num1 + num2));

    }

    public static void Main(string[] args){

        ABC obj = new ABC();

        obj.Sun1(68, 1);

        obj.Sun1(11, 12, 13);

    }

}
```

```
}
```

OUTPUT :-

```
Subtraction of Two Numbers : 67
Addition of 3 Numbers : 36
```

B) CODE :-

```
using System;
public class ABC{
    ABC(int num1, int num2){
        Console.WriteLine("Subtraction of Two Numbers : " + (num1 - num2));
    }
    ABC(int num, int num1, int num2){
        Console.WriteLine("Addition of 3 Numbers : " + (num + num1 + num2));
    }
    public static void Main(string[] args){
        ABC obj = new ABC(66, 7);
        ABC obj1 = new ABC(11, 12, 13);
    }
}
```

OUTPUT :-

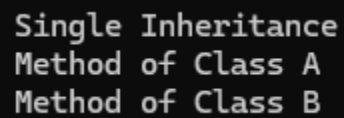
```
Subtraction of Two Numbers : 59
Addition of 3 Numbers : 36
```

C) - SINGLE INHERITANCE CODE :-

```
using System;
public class A{
    public void Fun1(){
        Console.WriteLine("Method of Class A");
    }
}
public class B : A{
    public void Fun2(){
```

```
        Console.WriteLine("Method of Class B");
    }
}

class SingleInheritance{
    public static void Main(string[] args){
        Console.WriteLine("Single Inheritance");
        B obj = new B();
        obj.Fun1();
        obj.Fun2();
    }
}
```

OUTPUT :-

```
Single Inheritance
Method of Class A
Method of Class B
```

- MULTILEVEL INHERITANCE CODE :-

```
using System;

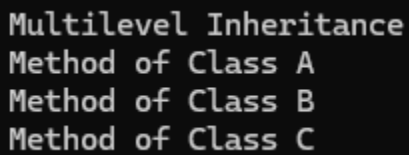
public class A{
    public void Fun1(){
        Console.WriteLine("Method of Class A");
    }
}

public class B : A{
    public void Fun2(){
        Console.WriteLine("Method of Class B");
    }
}

public class C : B{
    public void Fun3(){
        Console.WriteLine("Method of Class C");
    }
}
```

```
class MultilevelInheritance{  
    public static void Main(string[] args){  
        Console.WriteLine("Multilevel Inheritance");  
        C obj = new C();  
        obj.Fun1();  
        obj.Fun2();  
        obj.Fun3();  
    }  
}
```

OUTPUT :-



Multilevel Inheritance
Method of Class A
Method of Class B
Method of Class C

- HIERARICAL INHERITANCE CODE :-

```
using System;  
public class A  
{  
    public void Fun1(){  
        Console.WriteLine("Method of Class A");  
    }  
}  
public class B : A{  
    public void Fun2(){  
        Console.WriteLine("Method of Class B");  
    }  
}  
public class C : A{  
    public void Fun3(){  
        Console.WriteLine("Method of Class C");  
    }  
}  
class HierarchicalInheritance{  
    public static void Main(string[] args){  
        Console.WriteLine("Hierarchical Inheritance");  
        B obj1 = new B();  
        obj1.Fun1();  
        obj1.Fun2();  
  
        C obj2 = new C();  
        obj2.Fun1();  
        obj2.Fun3();  
    }  
}
```

}
OUTPUT :-

```
Hierarchical Inheritance
Method of Class A
Method of Class B
Method of Class A
Method of Class C
```

D) CODE :-

```
using System;

public interface A{
    void Fun1();
}

public interface B{
    void Fun2();
}

public class C : A, B{
    public void Fun1(){
        Console.WriteLine("Method of Interface A");
    }

    public void Fun2(){
        Console.WriteLine("Method of Interface B");
    }
}

class InterfaceDemo{
    public static void Main(string[] args){
        C obj = new C();

        obj.Fun1();
        obj.Fun2();
    }
}
```

OUTPUT :-

```
Method of Interface A
Method of Interface B
```

E)CODE :-

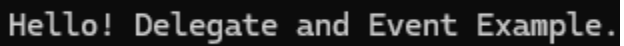
```
using System;

class Program{
    // Delegate
    public delegate void Message();
```

```
// Event
public static event Message ShowMessage;

static void Main(){
    ShowMessage += Display;
    ShowMessage();
}

static void Display(){
    Console.WriteLine("Hello! Delegate and Event Example.");
}
}
```

OUTPUT :-**F) CODE :-**

```
using System;
class Program
{
    static void Main()
    {
        try
        {
            int a = 10;
            int b = 0;
            int c = a / b;

            Console.WriteLine(c);
        }
        catch (DivideByZeroException)
        {
            Console.WriteLine("Cannot divide by zero.");
        }
    }
}
```

OUTPUT :-